

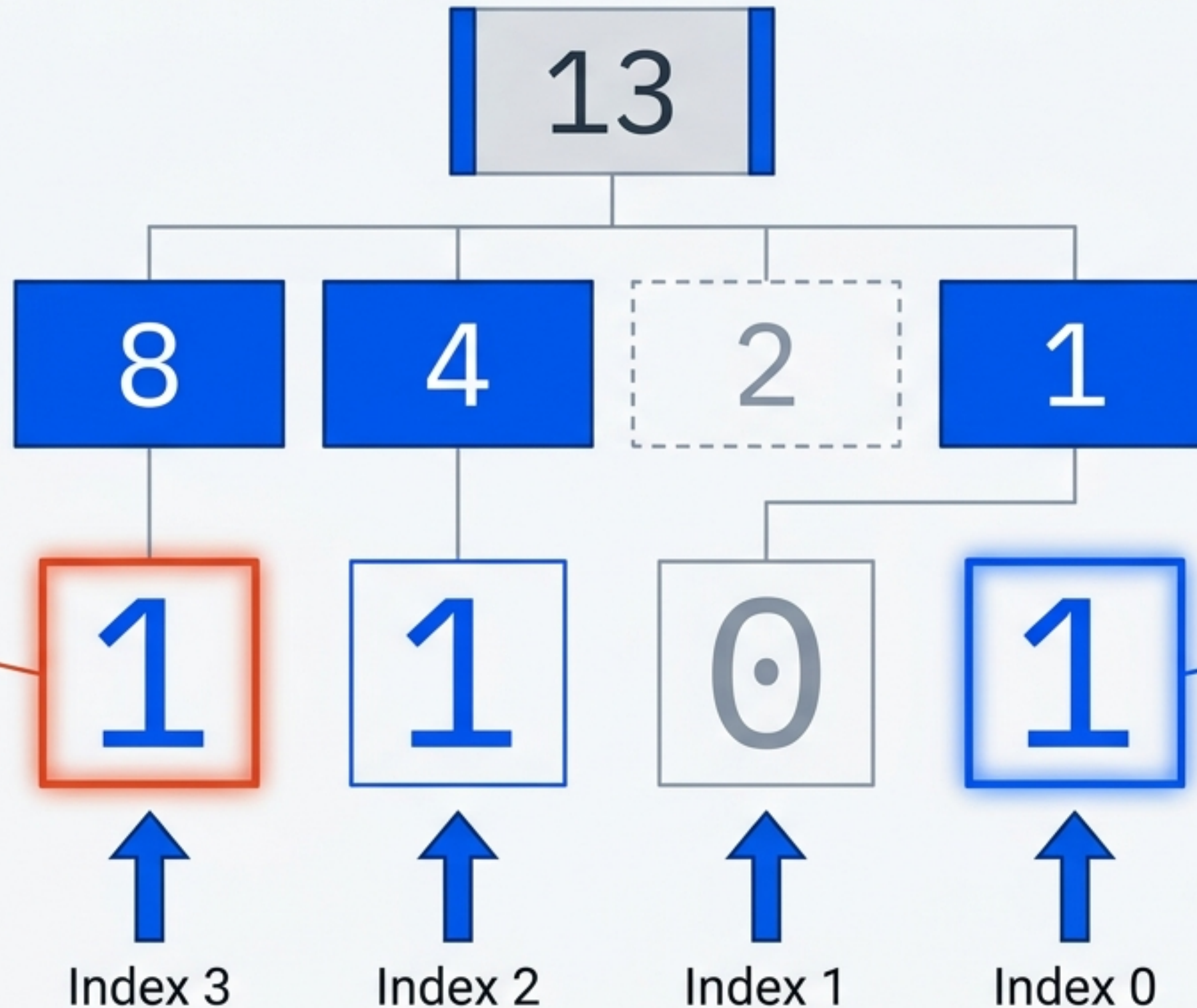
01010011 01100101 01100011 01110100 01101001 01101111 01101110
01000010 01101001 01110100 01110011 01101011 01101111 01101111
01000010 01101001 01110100 01101101 01100001 01101110 01101001
01110000 01110101 01101100 01100001 01110100 01101111 01101110
01010011 01101001 01100101 01101101 011 1 01101111 01101110
01000010 01101001 01110100 01101101 011 1 01101110 01101111
01000010 01101001 01110100 01101101 011 1 01101111 01101110
01000010 01101001 01110100 01101110 01100001 01110000 01101101
01000010 01101001 01110100 01101101 01110001 01101111 01101110

The Foundation of Bit Manipulation

A Visual Masterclass & Reference Guide

0(1) Space • Fast Boolean Checks • Foundational DSA Patterns

The Blueprint of a Binary Number



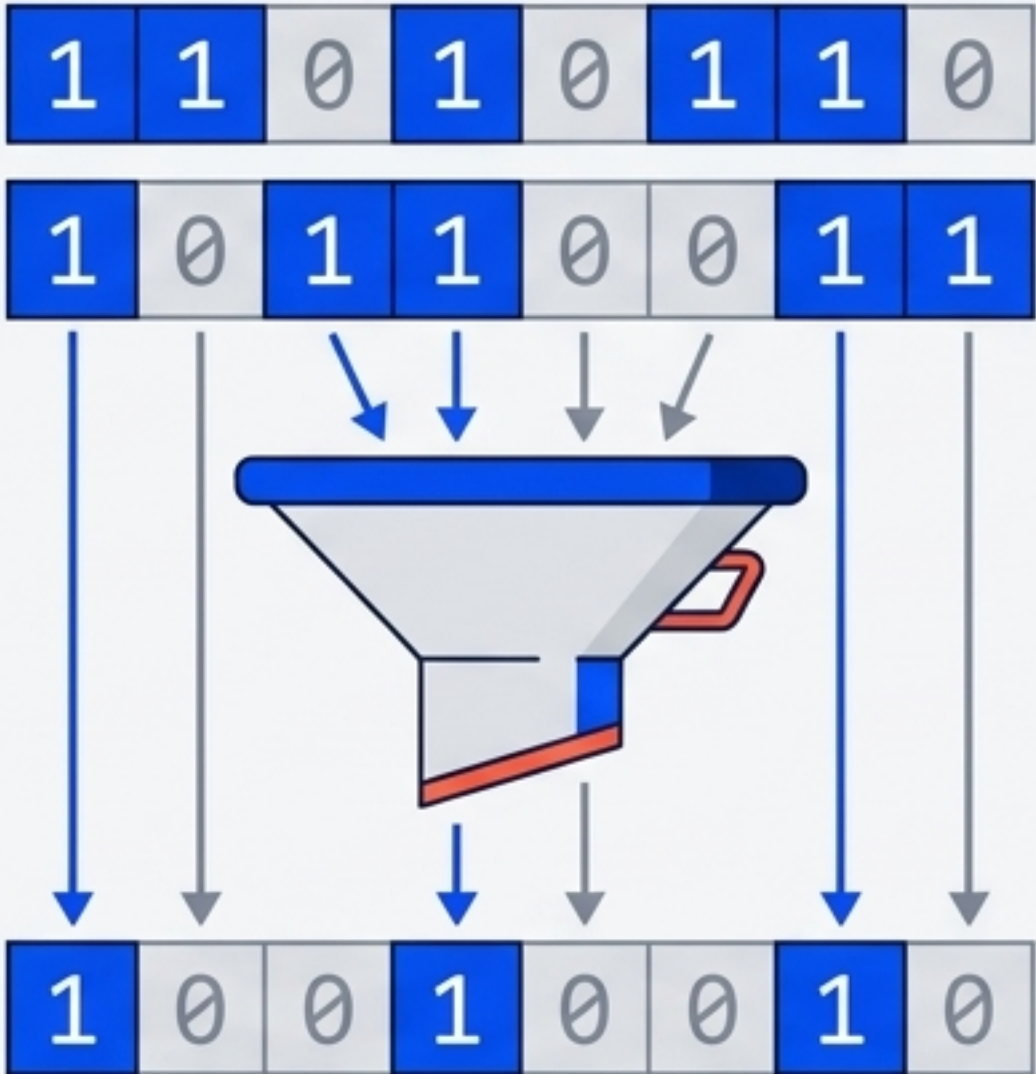
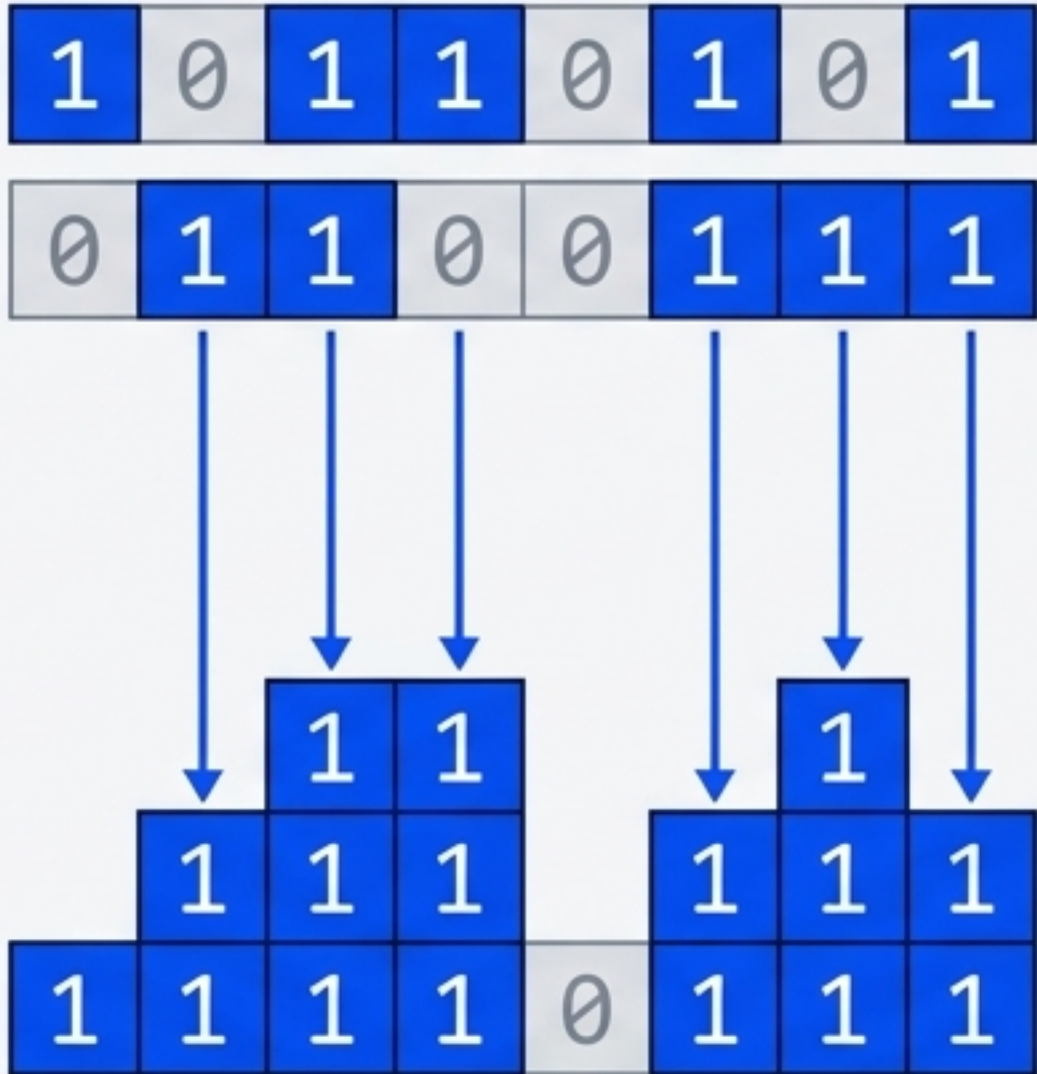
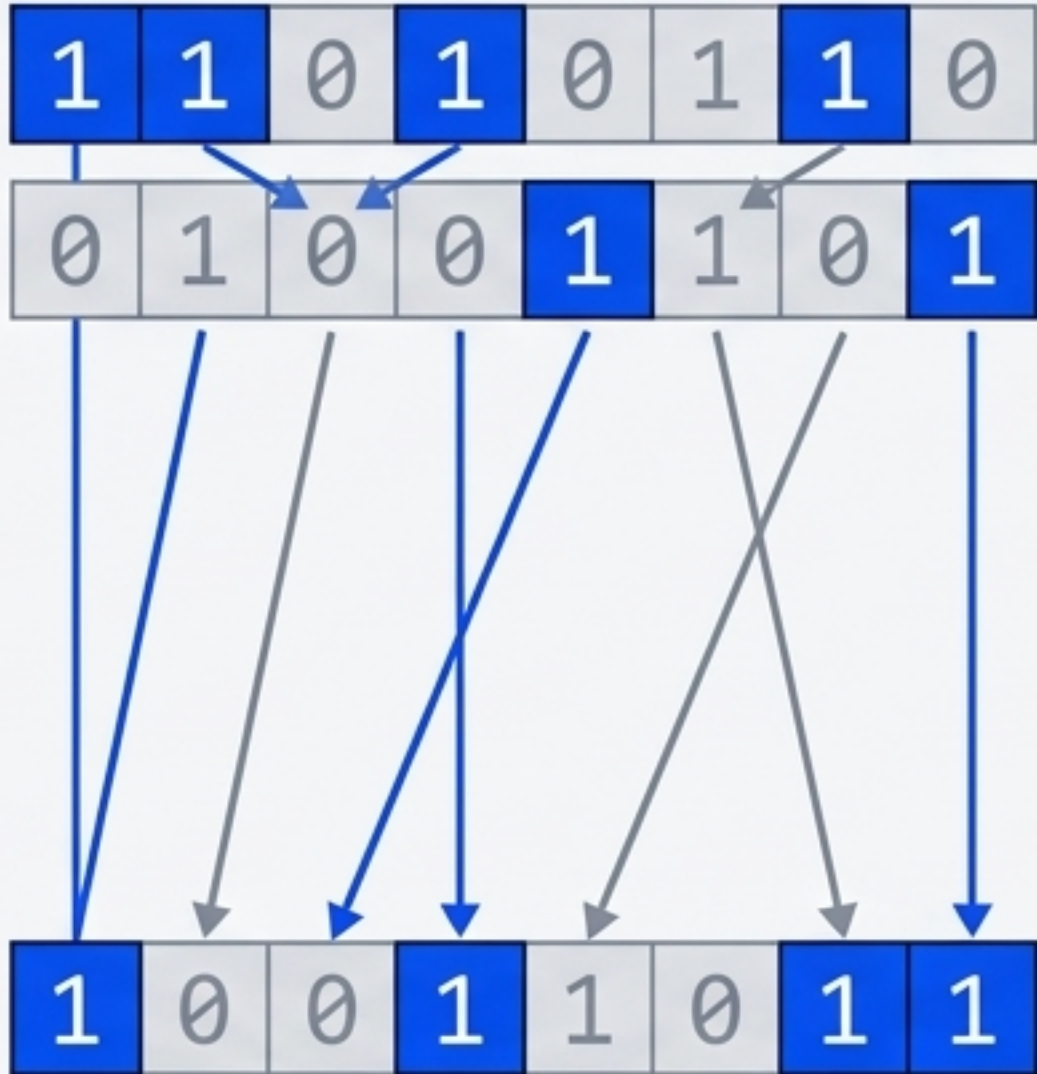
Most Significant Bit (MSB).

Represents the largest active power of 2.

Least Significant Bit (LSB).

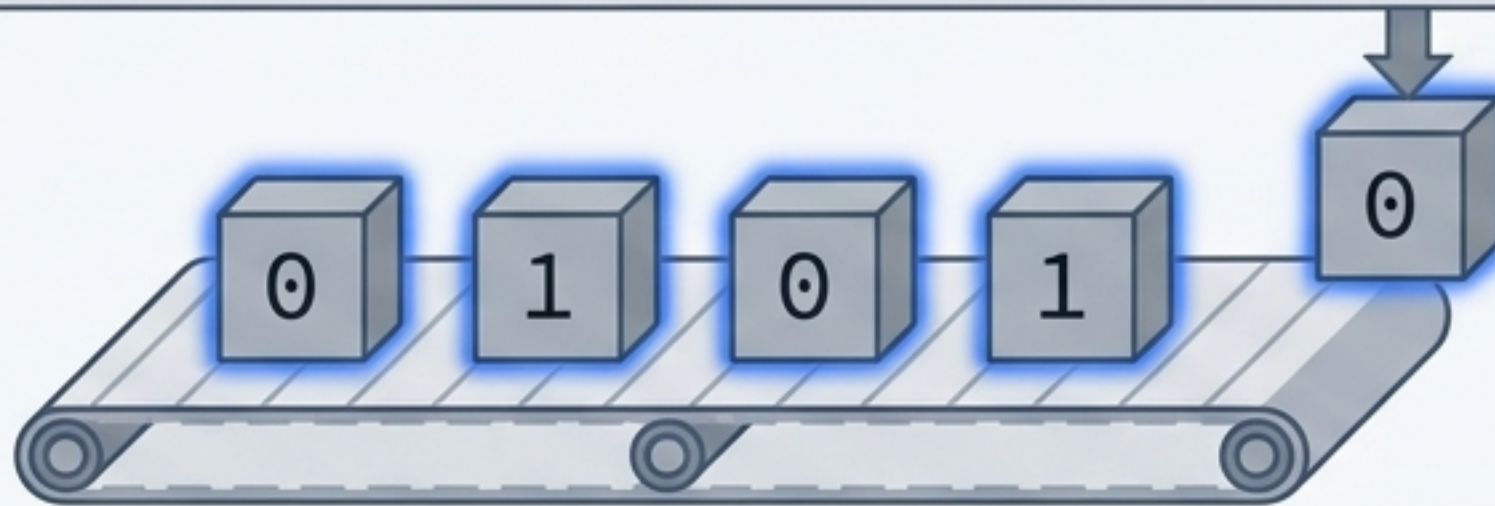
Represents the smallest value (1). Start counting index 0 here.

The Operator Matrix

AND (&) - The Filter Returns 1 only if BOTH bits are 1.	OR () - The Combiner Returns 1 if ANY bit is 1.	XOR (^) - The Difference Detector Returns 1 only if bits are DIFFERENT .
 Input 1: 1 1 0 1 0 1 1 0 Input 2: 1 0 1 1 0 0 1 1 Output: 1 0 0 1 0 0 1 0 Mental Model: A sieve letting only common bits through. Not addition.	 Input 1: 1 0 1 1 0 1 0 1 Input 2: 0 1 1 0 0 1 1 1 Output: 1 1 1 1 0 1 1 1 Mental Model: Stacking blocks to fill empty spaces. Used to turn bits ON.	 Input 1: 1 1 0 1 0 1 1 0 Input 2: 0 1 0 0 1 1 0 1 Output: 1 0 0 1 1 0 1 1 Mental Model: Highlighting contrasts and differences. Matched pairs cancel out.

Shift Operators

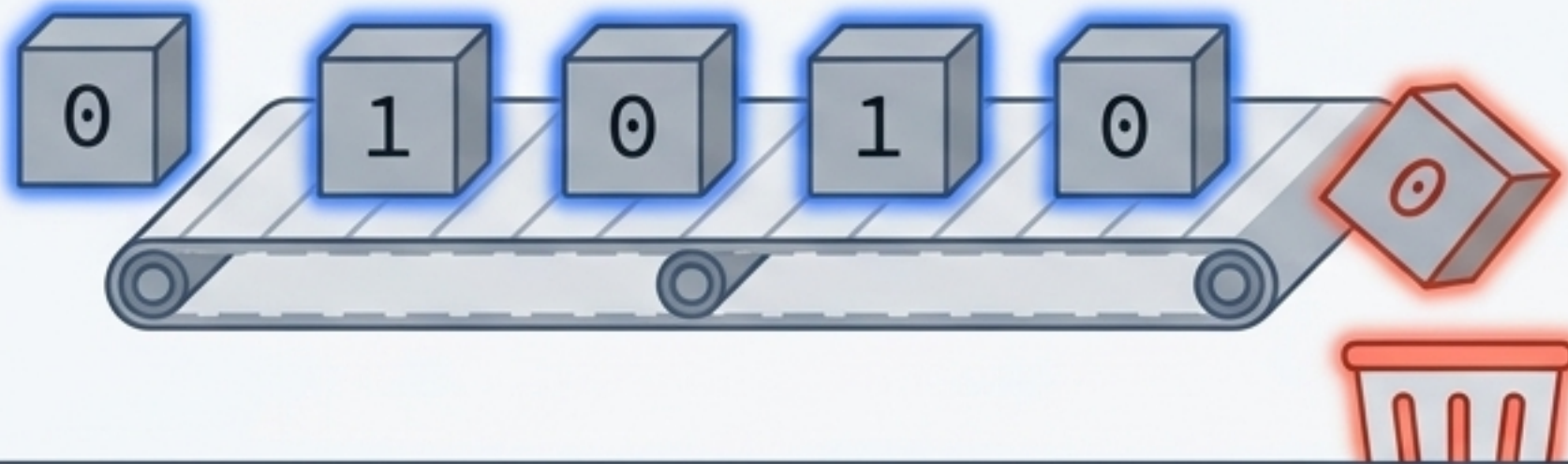
Left Shift (<<) - The Multiplier



$$n \ll k = n \times 2^k$$

Fast multiplication by powers of 2. Moves each bit to a position with double its previous value.

Right Shift (>>) - The Floor Divider

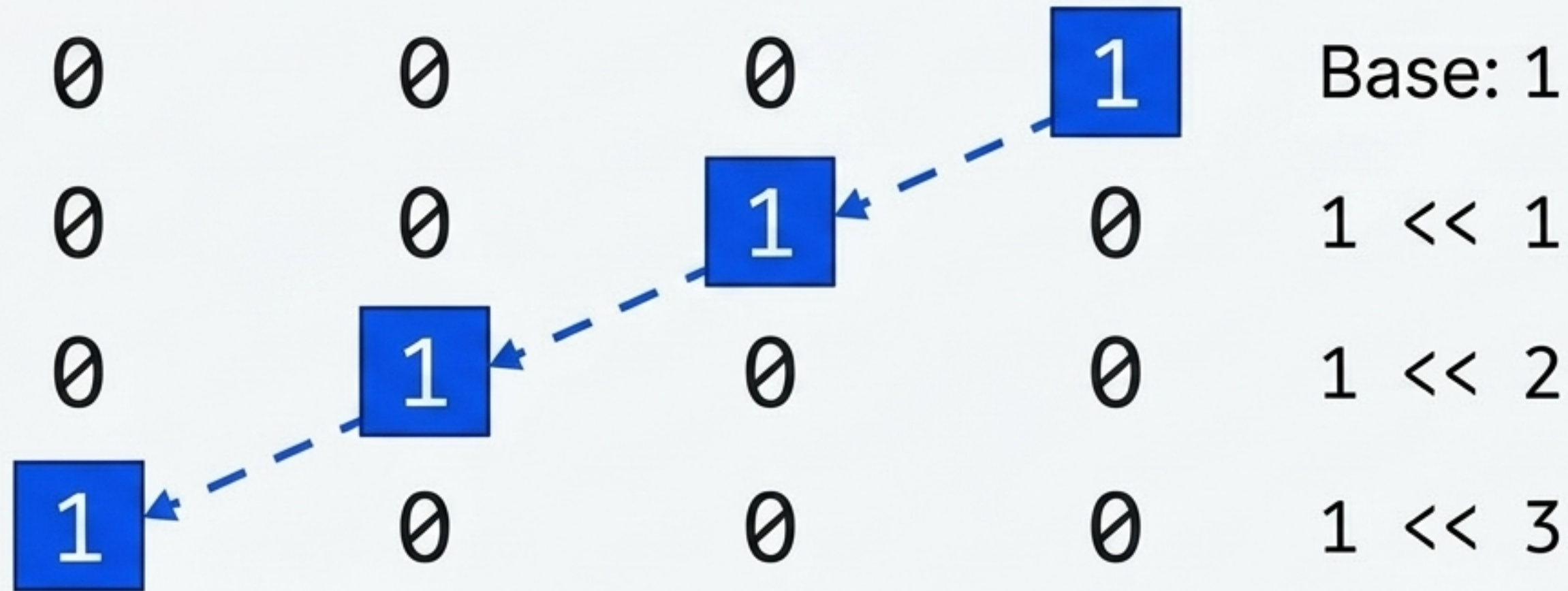


$$n \gg k = \text{floor}(n / 2^k)$$

Fast division. Any remainder is dropped off the edge (hence floor division).

The Mask Concept: $1 \ll i$

Synthesis



A **mask** is a targeted spotlight. It allows you to isolate a single bit position without disturbing the rest of the binary sequence.

Synthesis: By combining this wandering spotlight ($1 \ll i$) with our foundational operators ($\&$, $|$, $\wedge$), we unlock total control over any bit in a sequence.

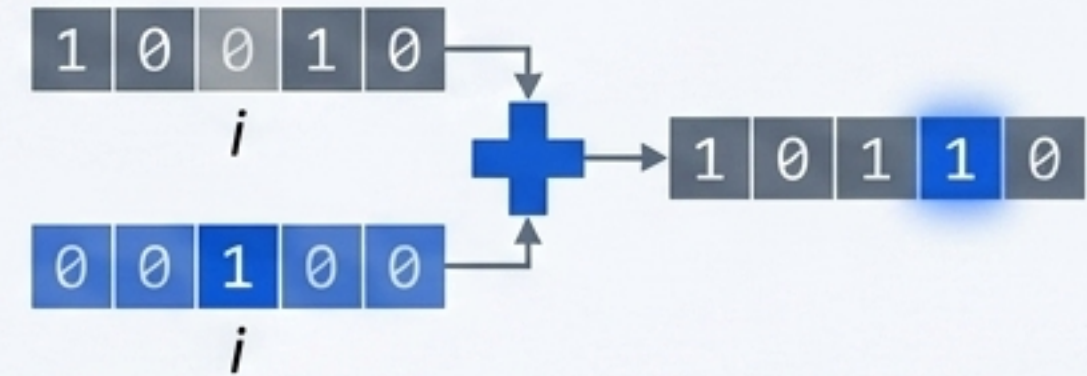
The 4 Core Actions

Check Bit $n \& (1 \ll i)$



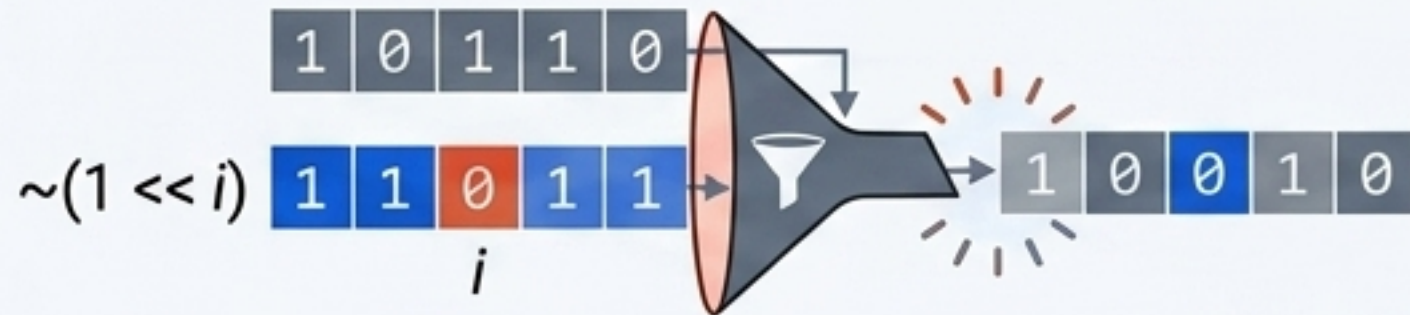
Mnemonic: AND isolates.

Set Bit $n \mid (1 \ll i)$



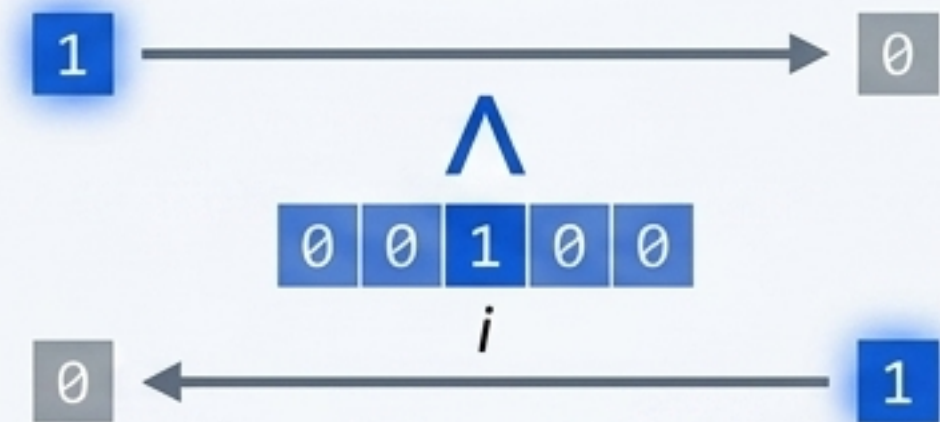
Mnemonic: OR turns ON.

Clear Bit $n \& \sim(1 \ll i)$



Mnemonic: AND + NOT snuffs out.

Toggle Bit $n \wedge (1 \ll i)$



Mnemonic: XOR flips.

Algorithm Spotlight: Brian Kernighan's Trick

$n \& (n - 1)$ instantly removes the rightmost set bit.

Step 1: Start with 13 (n)

1 1 1 0 1

Step 2: Subtract 1 (n - 1)

1 1 1 0 0

Step 3: AND the two

1 1 0 1 & 1 1 0 0

1 1 0 0

Subtracting 1 flips the rightmost "1" and everything after it.

Efficiency: Loop runs exactly k times where k is the number of set bits. $O(k)$ efficiency. Highly optimized for popcounts.

The Power of 2 Test

$$n > 0 \ \&\& \ (n \ \& \ (n - 1)) == 0$$

Power of 2 (e.g., 8)

1 0 0 0

Has exactly ONE set bit.

Subtract 1 yields `0 1 1 1`

	1	0	0	0
&	0	1	1	1
	0	0	0	0

Result is 0. Complete annihilation.

Non-Power of 2 (e.g., 10)

1 0 1 0

Has MULTIPLE set bits.

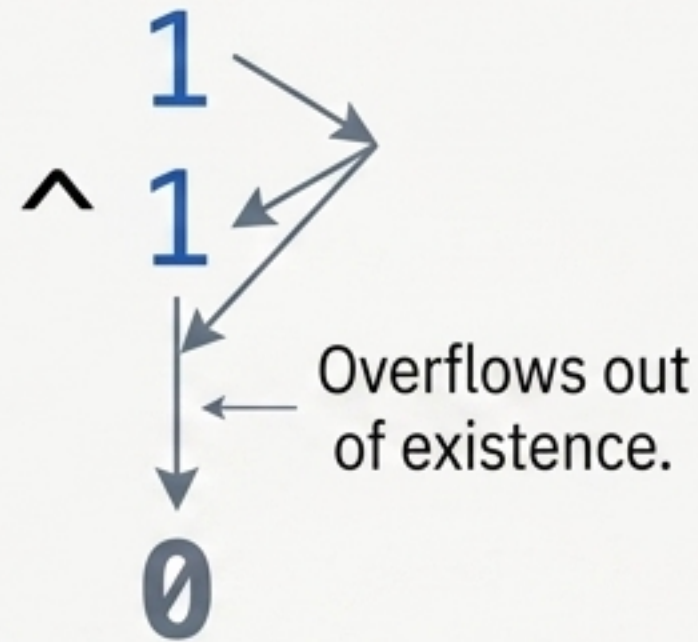
Subtract 1 yields `1 0 0 1`

	1	0	1	0
&	1	0	0	1
	1	0	0	0

Result is non-zero. Leftmost bit survives.

Crucial Rule: Why $n > 0$? Because 0 is not a power of 2, but $0 \ \& \ -1$ equals 0.

XOR: Addition Without Carry



Self-Canceling

$$a \wedge a = 0$$

Any number XORed with itself disappears entirely. Duplicates vanish.

Identity

$$a \wedge 0 = a$$

XOR with zero leaves the number completely untouched.

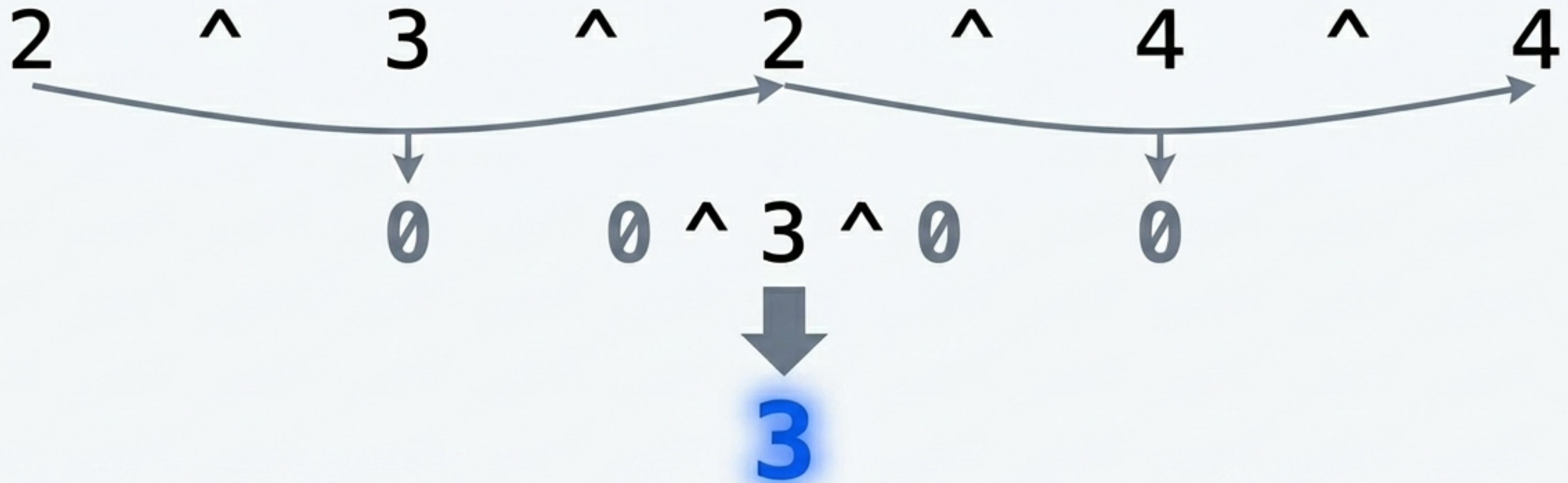
Reversible / Commutative

$$a \wedge b \wedge a = b$$

Order doesn't matter; matched pairs annihilate regardless of their sequence.

Applied XOR: The Duplicate Celler

Scenario: Array [2, 3, 2, 4, 4]. Find the element that appears exactly once.
 $O(1)$ space, $O(N)$ time.



Takeaway: XOR is a machine that **removes matched pairs**, keeps what is unmatched, and remembers only the difference.

Heuristic Decision Tree: When to use Bits

Does the problem involve...

Tracking Yes/No states or permissions?

Use Bitmasks

A single integer can track 32 separate boolean flags.

Generating all combinations of a small set (Subsets)?

Use Base-2 Counting

0 = not chosen,
1 = chosen.

Missing elements, unique elements, or duplicate cancellation in $O(1)$ space?

Use XOR

Math involving multiplication/division by powers of 2?

Use Shifts
($\ll$, $\gg$)

Think bit manipulation if the prompt says: toggle, odd/even, binary, flags, subset, $O(1)$ extra space.

Mental Traps & Pitfalls

The "AND is Addition" Myth

Error: $x \& y$ combines values.

Reality: It is a strict intersection filter. What bits do x and y share?
Example: $10 \& 6 = 2$.

Confusing Set vs. Toggle

Error: Using $\wedge$ when you want to ensure a bit is ON.

Reality: $|$ (OR) ensures it is ON.
 $\wedge$ (XOR) flips it blindly.

Indexing Left to Right

Error:

Thinking bit 0 is on the left side of the number.

Reality:

Always start index 0 on the LSB (the rightmost bit).

Forgetting Zero in Power of 2

Error:

$n \& (n-1) == 0$ is sufficient.

Reality:

Must include $n > 0$. $0 \& -1$ is 0 , but 0 is not a power of 2.

The One-Line Cheat Sheet

The Primitives		The 4 Core Actions		Advanced Algorithms	
&	Filter (Keep common 1s)	Spotlight Mask	$1 \ll i$	Count Set Bits	$n \& (n - 1)$ loop
	Combiner (Keep any 1s)	Check	$n \& (1 \ll i)$	Power of 2	$n > 0 \&\& (n \& (n - 1)) == 0$
^	Detector (Keep different bits)	Set	$n (1 \ll i)$	XOR Cancel	$a \wedge a = 0$
<<	Multiply by 2^k	Clear	$n \& \sim(1 \ll i)$	XOR Identity	$a \wedge 0 = a$
>>	Floor divide by 2^k	Toggle	$n \wedge (1 \ll i)$		

The absolute base layer of bit manipulation. End of Reference.